InfraNet Pro
NETWORK DOCUMENTATION

TECHNICAL MANUAL

Document your network the way you see it.

Visual mapping of your network infrastructure: floor plan, 19″ rack, L1/L2 topology, SNMP discovery, VLAN management and handover dossier — in a single application, installed on your own premises.

30
DEVICE TYPES

v1·v2c·v3
NATIVE SNMP DISCOVERY

16
THEMATIC CHAPTERS

On-premise · self-hosted | **Manual-first** | Documentation check (drift) | Label printing · **Avery / Dymo** | Bilingual IT / EN

Complete technical manual · One volume
Every feature of the app

Version: June 2026
Real app colours · A4-printable

Contents

A clear guide to InfraNet Pro's features — written for network engineers, system integrators and MSPs.

01 Requirements & installation

What you need to run InfraNet Pro and how to get it up and running in a few minutes.

InfraNet Pro is an **on-premise** application: it runs entirely on your own server or computer, with no cloud services, no external databases and no mandatory containers. Your data stays in-house.

What you need

Component	Requirement
System	Windows, Linux or macOS — any computer/server that stays on
Runtime	**Node.js 16 or later** (LTS recommended)
Browser	A modern browser: up-to-date Chrome, Edge or Firefox
Network	Reachability to the devices you want to document via **SNMP** (UDP 161) — only needed if you use automatic discovery
Disk space	Minimal: projects are lightweight files saved locally

Where to get Node.js

Download it for free from the official site **nodejs.org**: choose the **LTS** version (the most stable) for your operating system and install it with the guided package. On Linux you can also use your distribution's package manager (`apt`, `dnf`) or `nvm`. To check it is present: open a terminal and type `node -v` (it should respond with `v16` or higher).

No heavy infrastructure

No database (SQL/NoSQL), no cloud service, no Docker required. The app saves each project as a local file, with atomic writes and automatic backup copies: a power cut mid-save will not corrupt your work. *If you prefer containers, a ready-made Docker image is available too — see below.*

Installation in 4 steps

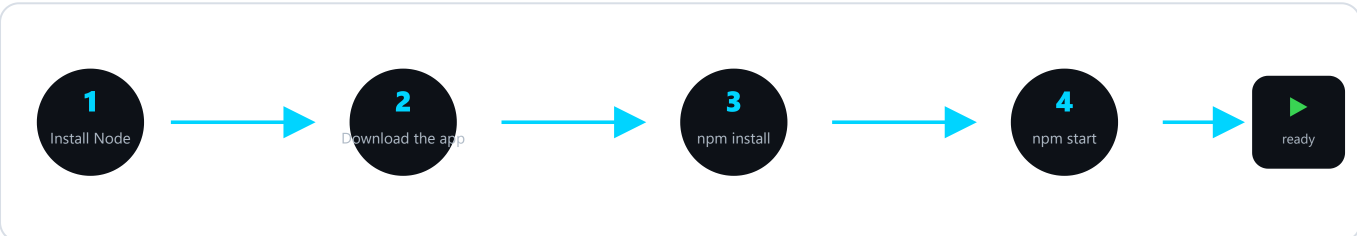

Fig. 1 — From package to first launch: four commands and the app is online.

Get the app in one of the two ways below: the first is handy if you'll update often, the second is the most immediate. Then — in **both cases** — install the dependencies and start.

Option A — with Git (recommended: updates become a single command). Requires Git installed; it downloads the project and tracks the version:

```
git clone https://github.com/muttley1973/infranetpro.git
cd infranetpro
```

Option B — download the ZIP (no Git needed). On the project page on GitHub press **Code → Download ZIP**, extract the folder and open it in a terminal. More immediate, but to update you'll have to download the ZIP again (see *Updating the app*).

Then, **in both cases**, install the dependencies (once only) and start. The app opens in your browser at the local address:

```
# install dependencies (once only)
npm install

# start the server
npm start

# open your browser at:
http://localhost:8421
```

> **Quick start on Windows**
> The folder contains `avvia.bat`: a double-click starts the server without opening a terminal. Handy for leaving the app running on a service workstation.

Alternatively: run with Docker

If you already use containers — Docker on your computer, or a NAS/server managed with **Portainer** — the project ships a ready-made `Dockerfile` and `docker-compose.yml`: the app builds and starts with a single command, and your data lives in a persistent volume that survives upgrades.

```
# set a fixed session key (once)
cp .env.example .env

# build and start in the background
docker compose up -d --build

# open your browser at:
http://<host-ip>:8421
```

The administrator password generated on first launch appears in the container log (`docker compose logs infranetpro`).

> **Network discovery & access**
>
> The `docker-compose.yml` uses **host networking** by default, so the container behaves like a native install — automatic **discovery sees the whole network** (MAC, vendor, SNMP) and the app is reachable at `http://<host-ip>:8421`. Since it is published on the host's interfaces (but login-protected), keep it on a trusted network; for outside access use a VPN or a reverse proxy. For an **isolated** instance (behind a reverse proxy, or on Docker Desktop where host networking is limited) use `docker-compose.bridge.yml`.

First login and configuration

- **Login.** On first launch you sign in with the default administrator account. The first thing to do is **change the password** from the user menu.
- **Configurable port.** The default port is `8421`; it can be changed via the `PORT` environment variable if already in use.
- **Access from other PCs.** Being a web server, once started it is reachable from other computers on the LAN by pointing to the server's IP (e.g. `http://192.168.1.10:8421`).
- **Where data lives.** Projects and floor plan background images are saved as files in the app's projects folder: a backup is simply a copy of that folder.

Updating the app

Your data — **projects, users and settings** — is stored in files separate from the application code: an update **does not touch it**. How you update depends on how you installed the app.

If you used Git (Option A) — just two commands in the app folder:

```
git pull
npm install
```

If you downloaded the ZIP (Option B) — download the new version from GitHub (**Code → Download ZIP**), extract it into a **new folder** and **bring your data back in**, copying it from the old folder:

Keep	What it holds
`projects/`	Your saved networks and floor-plan images
`users.json`	User accounts and passwords
`skins/`	Uploaded panel skins (if you use them)

Then run `npm install` in the new folder and restart. Do **not** copy the `node_modules` folder from the old one: it is regenerated by `npm install`.

Tip

Before updating, make a copy of the app folder: if anything goes wrong you can roll back in seconds. The **Git** method stays the most convenient if you update often.

02 InfraNet Pro at a glance

What it is, who it is for, and the principle that holds everything together: *manual-first*.

InfraNet Pro is designed to **draw and keep up to date** the documentation of a network: where devices physically are (the floor plan), how they are mounted in racks, how they are cabled and which VLANs they carry. It is built for people who know the network — engineers, system integrators, MSPs — and need documentation that is always accurate, not a drawing gathering dust in a drawer.

Two views of the same project

🗺 Map (physical)

Where devices are **physically** located: floor plan with rooms, positioned racks, wall sockets, access points. The reality you can touch with your hands.

🕸 Topology (logical)

How devices are **logically connected**: lines between devices derived from LLDP/CDP and switch forwarding tables, with filters for VLAN, trunk and wireless.

These are not two separate drawings: they are the **same graph** read from a physical or logical perspective. You switch between them with the `1` and `2` keys.

The principle: manual-first

> **The network suggests, the human decides**
>
> Data you enter manually is **never overwritten automatically**. SNMP discovery and consistency checking are *advisors*: they propose, flag, highlight — but the racks and cables you draw remain the authority. This is what makes the output trustworthy for a handover.

Many of the app's design choices follow from this: automatic discovery *fills in the gaps* but does not overwrite fields you have filled in; consistency checking shows you the differences but does not delete anything on its own; every alignment with reality is always your explicit decision, with a single click.

03 The interface

The toolbar, the three work areas and the shortcuts that speed everything up.

The screen is divided into three areas: on the left the element **library** to drag from, in the centre the **floor plan** and **rack** views, on the right the **properties** panel for whatever you select.

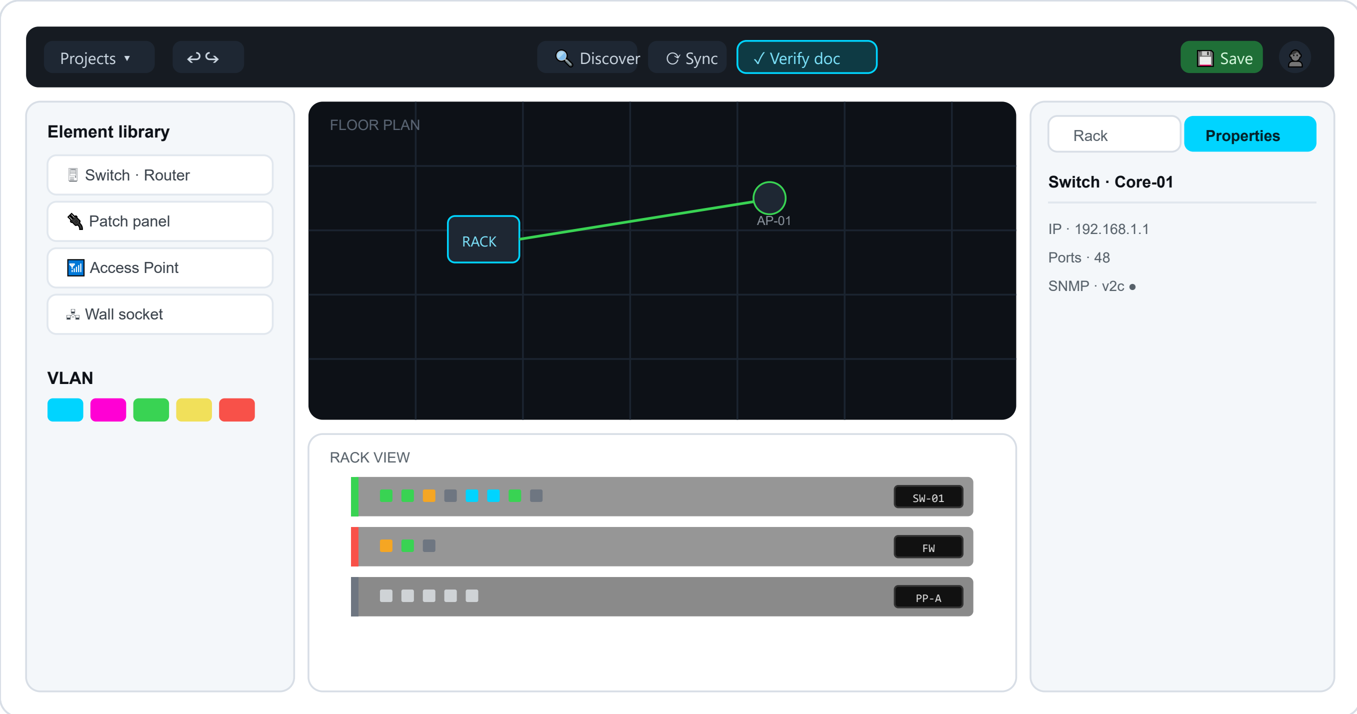

Fig. 2 — The three areas: library (left), dark floor plan + rack on white background (centre), properties panel (right). All **device data** (IP, ports, SNMP...) lives in the properties panel.

Essential shortcuts

Key	Action	Key	Action
`1` / `2`	Map / Topology view	`Ctrl+S`	Save project
`R` / `P`	Rack / Properties panel	`Ctrl+Z` / `Ctrl+Y`	Undo / Redo
`Space` +drag	Pan the map	`Del`	Delete selected element
`right-click`	Start / finish a port connection	`Esc`	Cancel connection / close panel

Connecting two ports — use the right mouse button

Cables are drawn with the **right mouse button**: right-click on the source port, then right- click on the destination port → the cable is created. `Esc` (or a click on empty space) cancels. For a *cross-rack* connection, a banner advises you: navigate to the other rack and right-click the destination port.

Tip

> After clicking a port, press `R` to jump to the connected rack without touching the mouse. And `Space`+drag is the quickest way to navigate large floor plans.

04 Device types

Over 30 ready-made types, split between rack appliances and floor plan endpoints.

Every element is dragged from the left-hand library. Types come pre-configured with an icon, port count and sensible attributes: a patch panel starts with 24 ports, a switch with 24, a server with a 2U height. Everything remains customisable.

Rack appliances

Type	Default	Type	Default
Switch	1U · 24 ports	Server	2U · 4 ports
Router	1U · 8 ports	Storage / NAS (SAN/RAID)	2U · 4 ports
Firewall	1U · 4 ports	UPS · PDU	power continuity / distribution
Patch panel	2U · 24 ports	KVM · Console server	out-of-band access
WLAN Controller	AP management	PBX / Phone system	telephony
Blank panel · Cable management	structural elements	Media converter	copper ↔ fibre

Floor plan endpoints

Type	Use	Type	Use
Wall socket	termination point (auto-named A-01...)	PC / Workstation	user workstation
Access Point	Wi-Fi coverage (auto-named AP-01...)	VoIP phone	telephony, often with cascaded PC
Network printer	endpoint with IP	Smartphone / Tablet	mobile endpoint (Wi-Fi / cellular, BYOD/MDM)
Webcam / CCTV	video surveillance (auto-named CAM-01...)	Desktop NAS	Synology/QNAP, SMB/NFS/iSCSI
IoT device	sensors, embedded controllers	Smart TV / Media player	AV / signage

Projector	meeting room	Badge reader	access control
Room/Area	resizable structural element	Custom endpoint	uncovered cases

Auto-naming

Wall sockets, access points and cameras number themselves as you drag them in: no need to rename fifty sockets one by one.

05 The floor plan

Set the physical scene: import the building plan, position devices, create rooms.

The floor plan is your physical canvas. You load the building plan as a background image, scale it to the real dimensions and place devices and rooms on top of it.

Importing and scaling the background

- You load an image (**PNG, JPG, GIF, WebP, SVG**) from the toolbar. PDF is not supported as a background: convert it to an image first.
- You adjust **scale** and **opacity** of the map from the properties panel, then press **Lock scale** to avoid accidentally moving it while you work.
- The **grid** aids alignment: devices snap every 20px (snap-to-grid). You can hide it — and with it hidden snap becomes free to the pixel.

Positioning and connecting

Drag devices from the library onto the correct areas of the building plan. A **click** selects, a **double-click** opens the full properties. Double-clicking a wall socket or an AP takes you directly to the rack of the connected cable, with the path highlighted.

> **Rooms and areas**
> A *Room/Area* is a resizable coloured rectangle (with an anti-move lock) that you use to draw offices, server rooms and departments. It always sits beneath other elements, like a background.

Navigation	How
Pan the view	Click+drag on empty area, or `Space` +drag anywhere
Zoom	Scroll wheel (centred on cursor) or `–` / `+` buttons · range 5%–500%
Deselect / clear VLAN filter	Click on empty area

06 The rack

The realistic 19″ cabinet front: units, ports, available capacity, gateway.

Every project can have multiple racks (from 6 to 60U, default 42U). The front panel is drawn realistically — grey chassis on a white viewport, faceplates with ports and LEDs — so the rack page looks like a real photo of the cabinet.

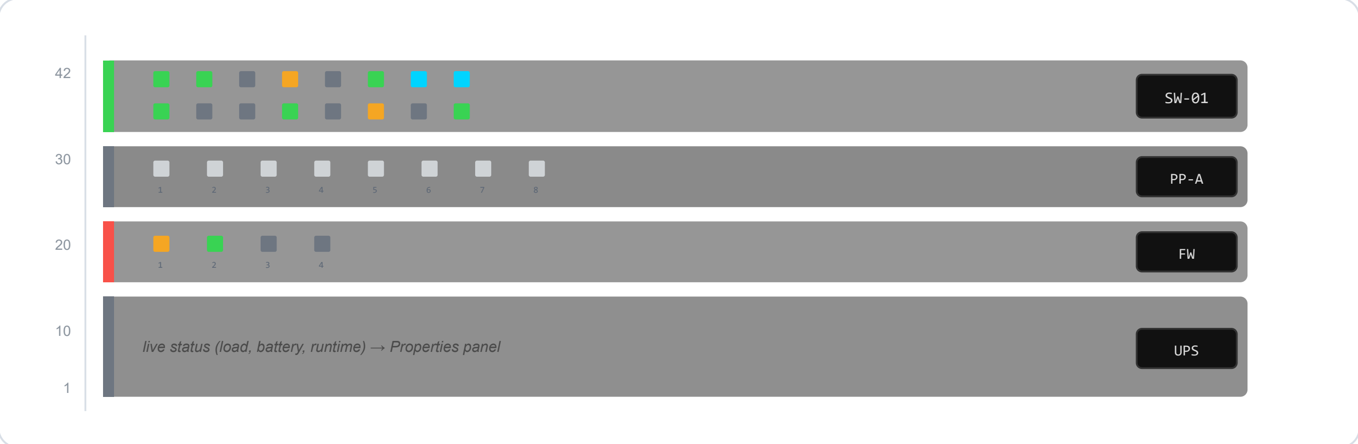

Fig. 3 — Rack front on white viewport. The **vertical stripe** on the left is the SNMP status (green ok · red error · grey not configured). Port LEDs are **small squares** (green active · amber idle · grey off · red error · blue LAG) with the **port number** below. Device data (IP, status, UPS runtime...) is read in the **Properties panel**.

Populating and configuring

- Drag appliances into the rack: they position themselves in the lowest free unit. Alternatively **import a CSV** and populate 50 appliances in a few seconds.
- The **port layout** (count, rows, separate SFP block) is adjusted from the properties panel; the front adapts from 8 to 48 ports while remaining readable.
- **Click** on a port = configure it; **right-click** = start a cable to another port (even on a different rack).

Free ports and L3 gateway

🟢 Free ports

Answers the everyday question: "*where do I plug in the new device?*". Highlights free ports **directly in the rack** (green = free, yellow = free on paper but seen active via SNMP) and produces a report with totals, exportable as CSV.

🌐 L3 map / Gateway

Promotes a VLAN's gateway from a mere IP to a **"VLAN → routing device" relationship**. An **L3** badge marks devices acting as gateways; a report lists VLANs, subnets, gateways and flags suspicious cases.

07 Cables, connections and wireless

Clear connections at a glance, the cable editor, the complete physical path and Wi-Fi "wave" associations.

A cable is created with the **right mouse button**: right-click on the source port, right-click on the destination port. The colour follows the VLAN; the **style** indicates how reliable that connection is.

The visual language of cables

Appearance	Meaning
Solid coloured line	**Manual** or **discovered** via LLDP/CDP — reliable
Animated dashed line	**Inferred** from MAC/ARP/FDB tables — to be confirmed (may be a transit link)
Sine wave	**Wireless connection** (radio association)

The cable panel: what you can set

Selecting a cable turns the right-hand panel into its **editor**. Here you declare what it is made of and what it carries — always following the *manual-first* principle: what you write by hand is authoritative.

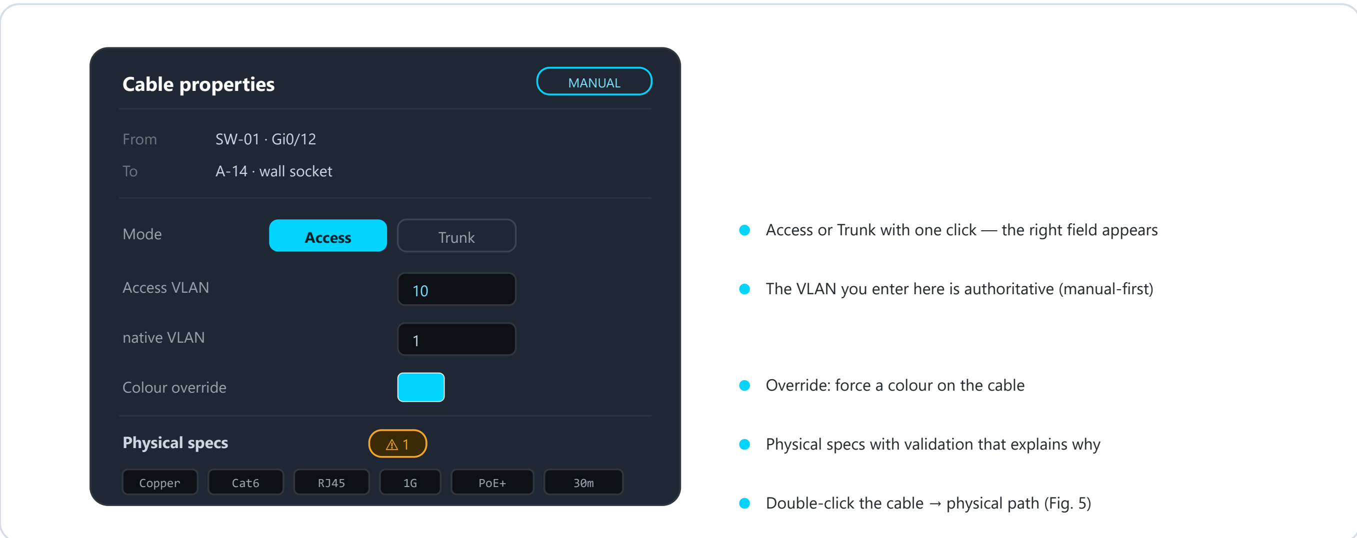

Fig. 4 — The cable editor in the Properties panel: endpoints (read-only), **Access/Trunk** mode, access VLAN and native VLAN, colour override, physical specs with the ⚠ warning badge, and (for radio links) the wireless toggle.

- **From / To** — the two endpoints (node and port), read-only.
- **Access or Trunk mode** — *Access* = a single VLAN (untagged); *Trunk* = multiple tagged VLANs (e.g. `10,20,100-200`). The right field appears depending on the choice.
- **VLAN** — the access VLAN (or the list of VLANs carried on the trunk) plus the editable **native VLAN** (PVID).
- **Colour override** — forces a colour for that cable, overriding the automatic VLAN colour.

- **Physical specs** — Medium, Category, Connector, Speed, PoE, Length, with the consistency validation described below.
- **Wireless connection** — the 📶 toggle turns the cable into a radio association (wave); with wireless enabled, the physical specs disappear.

The physical cable path

A real connection is often a **chain**: the switch reaches the patch panel, the patch panel reaches the wall socket, the socket reaches the endpoint. With a **double-click on a cable in topology** the app highlights *the entire physical path*, not just the clicked segment, and shows every segment in a single panel where you can select and edit each one individually.

Fig. 5 — **Physical path** highlighting (routing mode): endpoints in yellow, trace in cyan, unrelated branches dimmed. Press **Esc** to exit.

Smart validation (warnings, not blocks)

In the cable's physical specs the app checks that medium, category, connector, speed, PoE and length are consistent — and **explains why** when something does not add up:

- 10G on Cat5e → Cat6A required (Cat6 supports 10G only up to 55 m);
- copper run over 100 m → outside TIA-568 standard, use fibre;
- PoE on fibre → fibre does not carry power;
- medium↔connector inconsistencies (RJ45 on fibre, LC on copper).

These remain **warnings**: documentation stays free (manual-first), but you know immediately what and why.

Wireless: the radio port

> **"Wave" associations**
>
> Wi-Fi devices (APs, routers, firewalls) expose a 🔁 **antenna badge** 📶: it is the *radio port*, a virtual connector that hosts many clients without occupying physical ports. Drag a connection from the client to the radio badge and it is created as wireless, drawn as a wave. An AP can broadcast multiple SSIDs, each on a different VLAN: clients land automatically on the VLAN of their SSID, and the AP's wired uplink becomes a trunk carrying all those VLANs.

08 VLANs and colours

One colour per VLAN: cables colour themselves automatically and the network is readable at a glance.

You assign each VLAN a number, a name and a colour. Cables colour themselves automatically based on the VLAN they carry: a well-structured network becomes readable at a glance.

Default palette

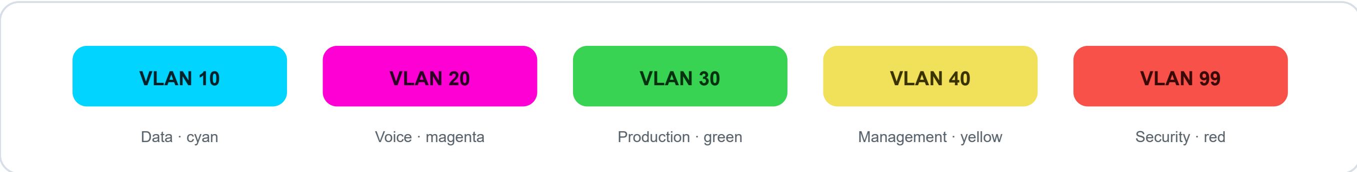

Fig. 6 — The five default VLAN colours. New VLANs receive a distinct colour automatically; every colour can be changed.

Access and Trunk

Mode	Meaning
Access	The port belongs to a single VLAN (untagged traffic). Typical for a PC, printer or camera.
Trunk	The port carries multiple VLANs (802.1Q-tagged): e.g. `10,20,100-200`. Typical for inter-switch uplinks and multi-SSID APs.

The app **propagates** VLANs automatically along cables and through pass-through elements (patch panels, wall sockets, media converters), so a run *switch → patch → socket → device* reflects the same trunk. The mode is a property of the active interface (the switchport), as in reality.

The VLAN filter

In topology, clicking a VLAN badge in the legend **isolates** that VLAN: everything else dims or disappears, and you see only where it goes. A double-click opens the member list. This is the quick way to verify that every VLAN is cabled where it should be.

09 Port states and presence

LEDs, the SNMP stripe and the dimming of devices that have disappeared from the network.

Status is read from colours, everywhere: on port LEDs in the rack, on device borders in the floor plan and in exports. They are the same four colours as the app.

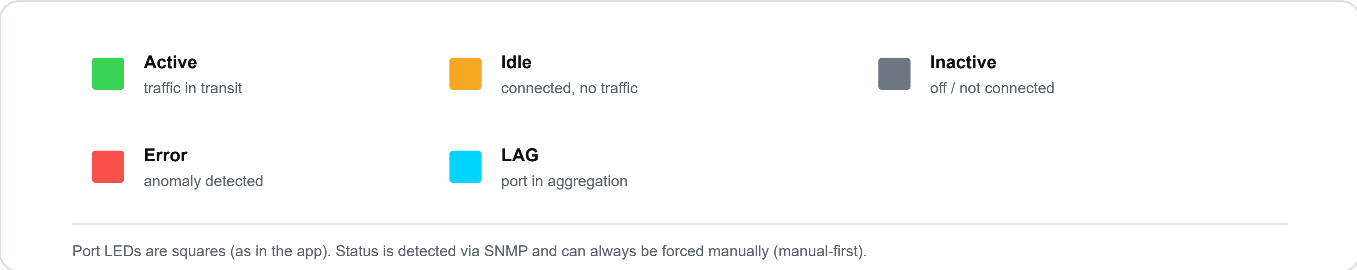

Fig. 7 — Port colour legend, identical in rack, floor plan and export.

The SNMP stripe ≠ presence

> **Two different signals**
>
> The **vertical stripe** on the left of the device indicates whether the last SNMP poll succeeded (green) or not (red/grey). This is different from *network presence*: an endpoint that does not speak SNMP (a PC, a camera) is always "grey" on the SNMP side, even if it is powered on and working.

Absent devices, dimmed on the map

When **Verify documentation** finds **no signal** from a device (no SNMP, no ping, no ARP, no trace in switch tables), that device is **greyed out** on the floor plan and in the rack: it remains clickable, but at a glance you understand that "it is on paper, but not on the network". When it comes back online, it lights up again automatically. Devices that are **Not verifiable** (on a subnet the check could not reach) are **not** greyed out: since their absence was never confirmed, they stay normal.

> **This is a Verify documentation feature**
>
> The dimming is based on the **full multi-signal sweep** (ping/ARP/TCP in addition to SNMP and forwarding tables) performed by **Verify documentation**: it is that process which determines absence and applies the greying.

10 SNMP: discovery and synchronisation

Letting the network speak for itself — with SNMP v1, v2c and v3, even without credentials on the first pass.

InfraNet Pro speaks **SNMP v1, v2c and v3** natively. Configure a device with a driver and address, and the app reads interfaces, VLANs, port status, MACs, and — on devices that expose them — printer toner levels, CPU/RAM, UPS and power continuity status.

Three actions that look similar but answer different questions

	🔍 Discover	⟳ Sync	✓ Verify doc
Question	"What is on the network that I have not yet drawn?"	"How are the devices I have drawn doing right now?"	"Does my drawing match reality?"
Starts from	a range/subnet	already configured IPs	already configured IPs
Finds new devices?	**Yes** (that is its purpose)	No (at most adds cables)	No (flags them, does not create them)
When	first mapping / new devices	quick refresh of status and topology	audit "is the doc still accurate?"

> **The most common confusion: Discover ≠ Sync**
>
> **Discover** starts from a *range* and looks for new hosts (active scan). **Sync** starts from the *IPs you already have* and refreshes them, also deriving cables from forwarding tables (FDB) and LLDP/CDP: you do not need "Discover" to get connections, **Sync already collects the FDB** and builds the topology on the fly, in one shot.

Discovery: scanning a subnet

1. You specify the subnet (e.g. `192.168.1.0/24`) and the SNMP credentials.
2. The app scans IP by IP, **classifies vendor and type** from the device profile (firewall, UPS, NAS, server, router, switch) and presents the candidates.
3. You select which ones to import: they enter the rack already with basic data filled in.

SNMP v3 detection works even **without credentials** on the first pass (it recognises the device and indicates that authentication is needed), and handles mixed-version environments in the same scan.

Automatic polling (data only)

From the **Network automations** popover, enable a background refresh at set intervals (1–30 min): it refreshes *only* the data (ports, VLANs, status) of SNMP devices, **without** touching the topology. This is the "lightweight" path; manual *Sync* remains the way to get updated connections as well.

UPS and power continuity units

UPS and ATS units have no interfaces to map: Sync queries them for dedicated parameters and populates a "Live status" block — mains/battery power feed, percentage and remaining runtime, load, voltages. UPS OIDs are standard (vendor-neutral); ATS OIDs are APC-specific and should be verified on your own hardware.

11 Automatic topology

The L1/L2 cross-check: the app draws discovered neighbours and proposes cables to create.

The Topology view reads the neighbours declared by devices (LLDP/CDP) and forwarding tables, and draws them as lines on the map. It is the **mirror** between the cabling you have documented and what the network reports about itself.

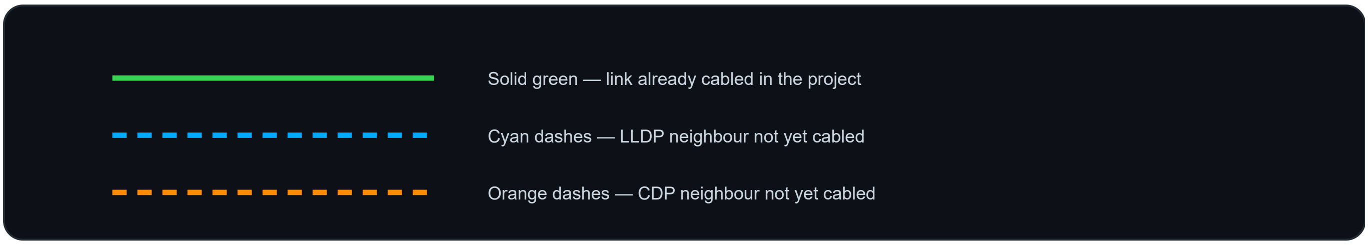

Fig. 8 — Topology lines. Clicking a dashed line opens the detail and the **"Create cable"** command, which matches ports by interface name.

Filters active only here

- **TRUNK** — highlights connections carrying multiple VLANs, dimming the rest.
- **ENDPOINT** — hides the last segment towards terminals: the view stops at wall sockets and the backbone (useful for reading the structure without the noise of PCs).
- **WLAN** — hides all wireless associations (waves), leaving only cables.

> **Create cable from suggestion**
>
> Let LLDP find the neighbours, then click "Create cable" on the dashed line: cable the network on the map without drawing every connection by hand.

12 The workflow

Not a straight line, but a *maintenance loop*: discover, document, verify.

You import the floor plan once. Then the *discover* → *document* → *verify* cycle runs over time: that is what keeps the documentation alive instead of letting it age.

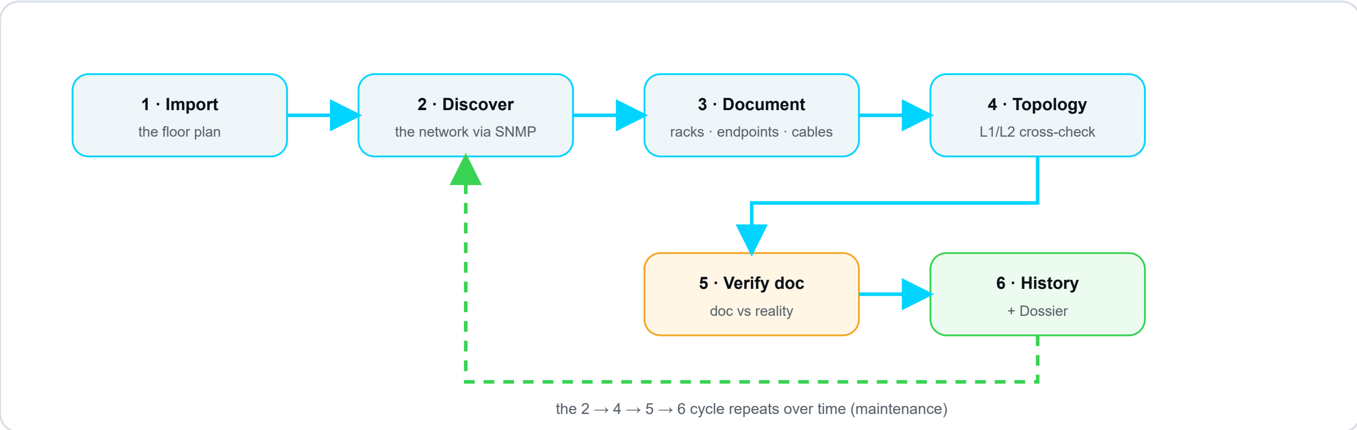

Fig. 9 — The maintenance loop. Phases 1–3 are the initial setup; **2 → 4 → 5 → 6** is the recurring cycle: after History/Dossier you return to **Discover** for the next round.

Where manual-first lives

- **In discovery** — fields you fill in manually stay yours: SNMP fills the gaps, it does not overwrite them.
- **In topology** — it is a mirror: it shows where cabling and reality agree or diverge, but it does not touch your cables.
- **In verification** — "Update doc" is an explicit action you take; the tool flags, it does not destroy.

13 Documentation check (Drift)

Stop hoping the doc is right: the app tells you exactly where it is not.

Documentation ages: a port moved during a fault, a VLAN changed on the fly, a cable unplugged and never updated. This divergence is called **drift**. The **Verify doc** button queries the real devices and **compares what is documented with what is actually there**, showing you only the differences.

> **In one sentence**
>
> It stops asking you "*I hope the doc is right*" and starts telling you "*here are the 4 points where the doc does not match reality*".

The seven report categories

Documentation check — 3 differences to review

🟩 **Consistent ports**	22
🟧 **State drift** `Core-01 / P12 vlan: 10 → 20`	1 [⟳] [🚫] [🔍]
IP change (same MAC)	0
Documented, absent from network	0
🟦 **On network, undocumented** `00:1A:2B:… seen on Core-01 · Gi0/24`	1
🟥 **Ghost cables**	1

Fig. 10 — The report organises differences by category. Each row has three actions: ⟳ **Update doc**, 🚫 **Ignore**, 🔍 **Investigate**.

Category	Meaning
🟩 **Consistent ports**	Doc and reality match (count only).
🟧 **State drift**	Documented port active but down, or different VLAN/speed.
IP change (same MAC)	Device alive at a different IP (DHCP lease changed) → update the IP in one click.
Documented, absent	No response to any signal (SNMP, ping, ARP, TCP) *and* its subnet was reached by the check → greyed out on the map.
Not verifiable (subnet not reached)	A documented device that is silent, but on a subnet the check could not reach (e.g. another VLAN the server cannot see): we do not declare it absent, because we never actually observed it.
🟦 **On network, undocumented**	A MAC appears on the switch but is not on the map → you can "add to map".

🟥 **Ghost cables**	Documented cable with port down across several consecutive checks (threshold: 3).

Multi-signal presence + BYOD noise

Presence is not judged by SNMP alone: every documented IP is also queried with **ping, ARP and TCP**, so PCs, IoT devices and UPS units do not end up among the "absent". And phones/laptops of users connecting to the guest Wi-Fi are **collected in a collapsed grey row** (identified by endpoint VLAN, busy uplinks or randomised MACs), keeping the list clean. Finally, a device is marked **absent** only if its subnet was actually reached by the check: those on out-of-reach subnets (another VLAN the server cannot see) land in **Not verifiable**, not among the absent.

Automatic IP renewal (opt-in)

You can enable automatic IP renewal from DHCP: the device's identity is its **MAC**, the IP is just an attribute that rotates. IPs set manually are *never* touched, and every renewal is logged in History.

14 Change history

Who changed what, and when. An append-only log that survives Undo.

On a documented network, multiple people work over time. Sooner or later the question comes: **"who changed this, and when?"**. **History** is the chronological log of project changes — an *append-only* diary: entries accumulate, they are never deleted.

> **In one sentence**
>
> It turns the map from a "snapshot of today" into a "film of how we got here".

What is recorded

Only **structural events**, not every minor tweak: device added/removed/renamed, cable created/removed, port VLAN changed, Sync and Verify run (with outcome), documentation aligned, project renamed. Every entry carries **date and time, user, action, object and detail**.

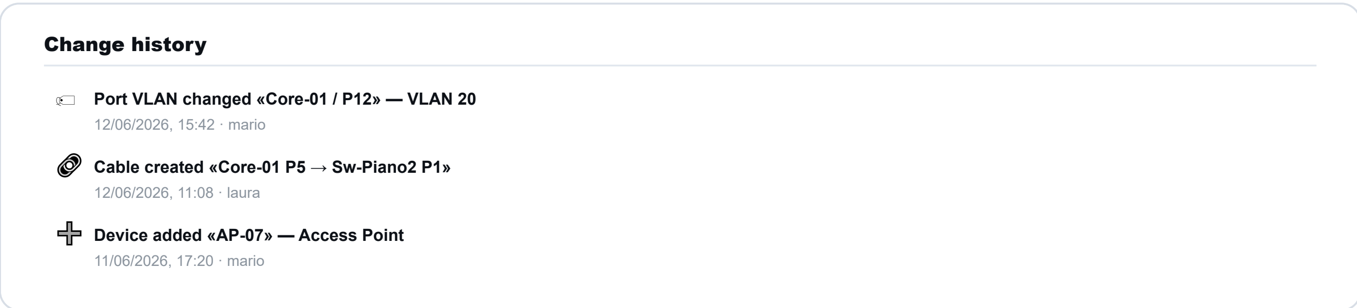

Fig. 11 — The log, most recent at the top. Filter by device/user/action and **export as CSV** for handovers and audits.

- **Cannot be undone.** Unlike everything else, History survives Undo/Redo: a log that can be erased would not be reliable.
- **Export as CSV** to attach documentary proof (handovers, audits, proof of work for MSPs).
- Only meaningful if **each person uses their own credentials**: with a single "admin" account, the "who" column loses its value.

15 Export, labels and Dossier

Getting documentation out of the app: SVG floor plan, cable labels, handover dossier.

Floor plan and backup

- **SVG** — the printable floor plan, with VLAN-coloured cables and device icons. If a VLAN filter is active, the SVG exports only that VLAN.
- **JSON** — the complete project backup, for moving it between installations.
- **CSV** — bulk import of tens of devices from an Excel/CMDB spreadsheet.

> **SVG = recommended format for the floor plan**
>
> The SVG export keeps the floor plan **vector**: you can enlarge it and print it at any size — even on a plotter or large format — **without losing sharpness**, unlike a raster image (PNG/JPG) that pixelates. It is the preferred format for archiving and delivering the map.

Printing cable labels

From the **Import/Export** → **Export labels** menu you generate cable labels ready to print, on standard market media:

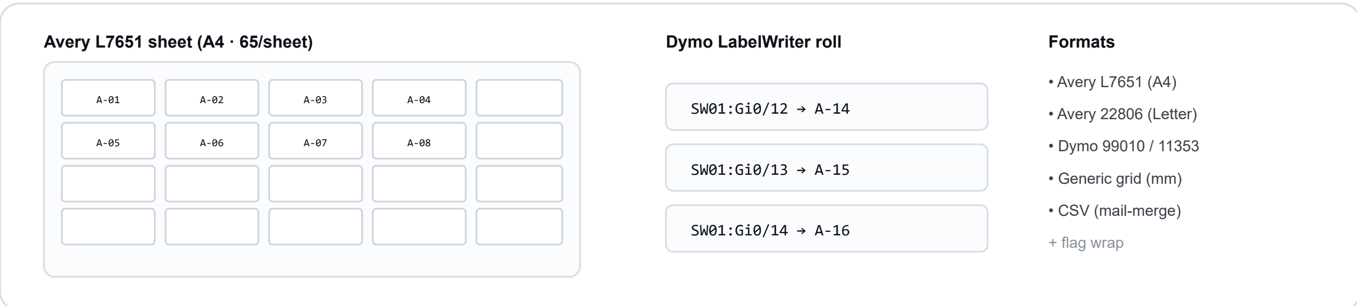

Fig. 12 — Labels on **Avery** sheets (L7651 A4, 22806 Letter), **Dymo** rolls (99010, 11353), generic grid configurable in mm, or CSV for mail-merge. Preview is live.

- Choose the **fields** to print (by default only the label ID) and the **flag wrap** option, which repeats the ID in both halves → readable from either side when wrapped around the cable.
- Geometries are nominal: do a **test print** and align it against the label sheet to check alignment; any offset can be corrected with the "generic grid".

Handover dossier

> **A single PDF, with one click**
>
> The **Handover dossier** collects everything the recipient needs: cover page with key figures, floor plan, rack fronts, cable inventory, as-built path trace, port assignment, VLAN/IPAM, notes and change history. For an MSP it is a *billable deliverable*; for a company it is *insurance against staff turnover*. Generated from the Import/Export menu.

For a partial PDF (floor plan only, rack only...) there is the classic **Export PDF** with checkboxes to choose individual sections. Tip: run a *Sync* just before exporting, so ports, VLANs and status in the dossier are up to date.

16 Bilingual IT / EN

Interface in Italian or English, with networking terms always kept in their original form.

The interface is **bilingual Italian/English**. The selector is in the user menu; the choice is remembered between sessions. You switch between languages without reloading and without losing your work.

> **The technical glossary is not translated**
>
> Networking terms — **VLAN, SNMP, LLDP/CDP, SFP, trunk, uplink** — and vendor names remain unchanged in both languages: translating them would only cause confusion. What is translated is the interface (menus, panels, messages), not the language of the trade.

Bilingual coverage is verified automatically: a check flags any string translated on only one side, so both languages stay aligned as the app grows.

Support the project

InfraNet Pro is **free and open source**. If it saves you time, you can buy the development a coffee: every contribution helps fund new features, fixes and support.

> ☕ **Buy me a coffee**
>
> Support page: **ko-fi.com/infranetpro**